

Building Perceptron

Réponses rédigées — MSc IA & Data, La Plateforme

Lucien Nzeutom

1. Initiation au Deep Learning

Machine Learning

Le Machine Learning (apprentissage automatique) est une branche de l'intelligence artificielle dans laquelle un programme apprend à partir de données au lieu de suivre des règles écrites à la main. On lui donne des exemples, il en tire un modèle, et ce modèle sert ensuite à prédire ou classer de nouveaux cas. Exemples d'algorithmes : régression linéaire et logistique, arbres de décision, Random Forest, SVM, k plus proches voisins.

En ML « classique », c'est souvent au data scientist de choisir ou de construire les variables utiles (feature engineering) avant l'entraînement.

Deep Learning

Le Deep Learning (apprentissage profond) est une sous-partie du Machine Learning. Il utilise des réseaux de neurones artificiels composés de plusieurs couches empilées. Chaque couche transforme les données et apprend une représentation un peu plus abstraite que la précédente : sur une image, les premières couches repèrent des bords, les suivantes des formes, les dernières des objets. Le modèle apprend donc lui-même les variables utiles, sans qu'on ait besoin de les définir à la main.

Comparaison

Critère	Machine Learning classique	Deep Learning
Type de données	Données tabulaires (tableaux, bases de données)	Images, son, texte, vidéo
Volume nécessaire	Quelques centaines à quelques milliers de lignes	Beaucoup de données (souvent des millions)
Variables	Choisies ou construites à la main	Apprises automatiquement
Puissance de calcul	Un CPU suffit, entraînement rapide	GPU souvent nécessaire, entraînement long
Interprétabilité	Plutôt bonne	Faible (effet « boîte noire »)

Quand utiliser l'un plutôt que l'autre ?

J'utilise le Machine Learning classique quand les données sont tabulaires, de taille moyenne, et quand il faut pouvoir expliquer la décision (santé, banque, assurance). C'est le cas du dataset Breast Cancer de ce projet : 569 lignes et 30 variables numériques.

Le Deep Learning devient intéressant quand les données sont non structurées (images, texte, audio), très nombreuses, et quand la performance compte plus que l'explication. Il demande aussi plus de temps et de ressources de calcul.

Trois applications du Deep Learning

1. La vision par ordinateur en imagerie médicale. Des réseaux de neurones convolutifs (CNN) analysent des radios, des mammographies ou des IRM pour repérer des tumeurs ou des anomalies. Ils servent d'aide au diagnostic pour les radiologues.

2. Le traitement du langage naturel. Les modèles de type Transformer, comme ChatGPT d'OpenAI, génèrent du texte, répondent à des questions, résument ou traduisent. Ils sont entraînés sur d'énormes quantités de texte.

3. La reconnaissance de dessins et la génération d'images. L'expérience Quick, Draw! de Google AI Experiments reconnaît en temps réel ce que l'utilisateur dessine, grâce à un réseau entraîné sur des millions de croquis. Dans la même famille, des modèles comme DALL·E d'OpenAI génèrent des images à partir d'une description en texte.

2. Dessine-moi un neurone artificiel

Q1. Qu'est-ce qu'un perceptron ? Quel lien avec le neurone biologique ?

Le perceptron est le premier modèle de neurone artificiel, proposé par Frank Rosenblatt en 1958. C'est un classifieur binaire : il reçoit plusieurs valeurs en entrée et décide entre deux classes (0 ou 1). Il sépare les données à l'aide d'une droite, ou d'un hyperplan quand il y a plus de deux variables.

Il s'inspire du neurone biologique :

Neurone biologique	Perceptron
Les dendrites reçoivent des signaux	Les entrées $x_1, x_2, \dots, x_n$
Les synapses renforcent plus ou moins chaque signal	Les poids $w_1, w_2, \dots, w_n$
Le corps cellulaire additionne les signaux	La somme pondérée $w \cdot x + b$
Le neurone ne s'active que si un seuil est dépassé	La fonction d'activation (seuil)
L'axone transmet le signal de sortie	La sortie $\hat{y}$ (0 ou 1)

Q2. Fonction mathématique du perceptron et usage

Le perceptron calcule une somme pondérée de ses entrées, ajoute un biais, puis applique une fonction d'activation :

$$\hat{y} = \varphi(w_1x_1 + w_2x_2 + \dots + w_nx_n + b) = \varphi(w \cdot x + b)$$

- **$x = (x_1, \dots, x_n)$** : les variables d'entrée d'un exemple.
- **$w = (w_1, \dots, w_n)$** : les poids, qui donnent l'importance de chaque variable. Ce sont eux que le modèle apprend.
- **b** : le biais, qui décale le seuil de décision (il permet à la droite de ne pas passer par l'origine).
- **$z = w \cdot x + b$** : le score, ou entrée nette.
- **φ** : la fonction d'activation, qui transforme le score en classe.
- **$\hat{y}$** : la prédiction, 0 ou 1.

Usage : classer des exemples en deux catégories quand ces catégories peuvent être séparées par une frontière linéaire (par exemple : tumeur maligne ou bénigne, spam ou non spam).

Q3. Règles d'apprentissage

Règle du perceptron (Rosenblatt). Après chaque exemple, on compare la prédiction $\hat{y}$ à la vraie classe y et on corrige les poids :

$$w_i \leftarrow w_i + \eta (y - \hat{y}) x_i \quad b \leftarrow b + \eta (y - \hat{y})$$

η (êta) est le taux d'apprentissage, un petit nombre comme 0,01 ou 0,1 qui règle la taille des corrections. Si la prédiction est juste, $(y - \hat{y})$ vaut 0 et les poids ne bougent pas. Si le modèle prédit 0 au lieu de 1, les poids augmentent dans la direction de x ; s'il prédit 1 au lieu de 0, ils diminuent.

Règle delta / descente de gradient (Adaline). Au lieu de corriger seulement en cas d'erreur, on définit une fonction de coût continue, par exemple l'erreur quadratique moyenne, et on ajuste les poids dans le sens opposé à son gradient : $w \leftarrow w - \eta \nabla J(w)$. Cette règle converge même quand les classes ne sont pas parfaitement séparables et prépare le terrain aux réseaux de neurones modernes.

Q4. Fonction d'activation

Le perceptron classique utilise la fonction de Heaviside, ou fonction échelon (seuil) :

$$\varphi(z) = 1 \text{ si } z \geq 0, \varphi(z) = 0 \text{ sinon}$$

Elle donne directement une classe. Sa limite est qu'elle n'est pas dérivable, ce qui empêche d'utiliser la descente de gradient. Les modèles plus récents la remplacent par la sigmoïde, qui renvoie une probabilité entre 0 et 1, ou par ReLU dans les réseaux profonds.

Q5. Processus d'entraînement

1. Initialiser les poids et le biais à zéro ou à de petites valeurs aléatoires.
2. Pour chaque exemple du jeu d'entraînement : calculer $z = w \cdot x + b$, puis la prédiction $\hat{y} = \varphi(z)$.

3. Comparer $\hat{y}$ à la vraie classe y et mettre à jour w et b avec la règle du perceptron.
4. Un passage complet sur toutes les données s'appelle une époque. Répéter les époques jusqu'à ce qu'il n'y ait plus d'erreur ou qu'on atteigne le nombre maximal d'époques fixé.
5. Suivre le nombre d'erreurs à chaque époque pour vérifier que le modèle apprend (courbe d'apprentissage).

Avant l'entraînement, il faut standardiser les variables, car le perceptron est sensible à leur échelle.

Q6. Limites du perceptron

- **Séparation linéaire uniquement** : il ne sait tracer qu'une frontière droite. Il échoue sur des problèmes non linéaires comme la fonction XOR, ce qu'ont montré Minsky et Papert en 1969.
- **Pas de convergence garantie** : si les classes ne sont pas linéairement séparables, les poids continuent d'osciller sans se stabiliser.
- **Sortie binaire sans probabilité** : il dit « 0 » ou « 1 » sans niveau de confiance, ce qui est gênant en santé.
- **Deux classes seulement** : pour plus de classes, il faut combiner plusieurs perceptrons.
- **Sensibilité à l'échelle et à l'ordre des données** : il faut standardiser les variables et mélanger les exemples.

Q7. Mon perceptron en Python

J'ai codé une classe Perceptron en programmation orientée objet, dans le fichier perceptron.py. Elle contient :

- **__init__(learning_rate, n_epochs)** : les hyperparamètres.
- **fit(X, y)** : initialise les poids et applique la règle d'apprentissage sur plusieurs époques, en enregistrant le nombre d'erreurs par époque.
- **net_input(X)** : calcule $w \cdot x + b$.
- **predict(X)** : applique la fonction seuil et renvoie 0 ou 1.

Je l'ai testée sur des données factices générées aléatoirement : deux nuages de points bien séparés. Le modèle trouve une droite qui les sépare et le nombre d'erreurs tombe à zéro en quelques époques. Je l'ai aussi testée sur la porte AND (réussite) et sur la porte XOR (échec), ce qui illustre la limite vue en Q6.

[À compléter avec mes résultats : nombre d'époques nécessaires, η choisi, capture de la frontière de décision.]

3. Breast Cancer Wisconsin

Les données

Le dataset Breast Cancer Wisconsin (Diagnostic) contient 569 patientes et 30 variables numériques calculées à partir d'images de ponctions de masses mammaires. La cible est le diagnostic : maligne (M, 212 cas, 37 %) ou bénigne (B, 357 cas, 63 %).

Les 30 variables décrivent les noyaux cellulaires avec 10 mesures (rayon, texture, périmètre, aire, lissage, compacité, concavité, points concaves, symétrie, dimension fractale), chacune déclinée en trois versions : la moyenne, l'erreur standard et la pire valeur (« worst »). Le dataset ne contient pas de valeurs manquantes.

J'ai codé la cible avec 1 = maligne et 0 = bénigne, car c'est la classe maligne qu'on cherche à détecter.

Analyse exploratoire

[À compléter après l'EDA : distribution de la cible, variables qui séparent le mieux les deux classes, corrélations fortes (rayon, périmètre et aire sont presque redondants), outliers et décision prise.]

Réduction de dimensionnalité

J'ai choisi l'Analyse en Composantes Principales (ACP). Plusieurs variables sont très corrélées entre elles, et l'ACP résume cette information redondante dans un petit nombre de nouveaux axes, les composantes principales, en gardant le maximum de variance. Elle permet aussi d'afficher les données en 2D.

Je l'applique après avoir séparé train et test et après standardisation (le StandardScaler et l'ACP sont ajustés sur le train uniquement, pour éviter une fuite de données). Je garde le nombre de composantes qui explique environ 95 % de la variance.

Limite de ce choix : les composantes ne correspondent plus à des variables médicales réelles, ce qui rend le modèle moins interprétable. Boruta, qui sélectionne les variables d'origine les plus utiles en les comparant à des copies mélangées, serait une alternative plus lisible.

[À compléter : nombre de composantes retenues et part de variance expliquée.]

Choix des métriques

Sur un problème de santé, les deux erreurs n'ont pas le même coût. Un faux négatif (dire « bénigne » à une patiente qui a une tumeur maligne) est bien plus grave qu'un faux positif, qui mène à des examens supplémentaires. L'accuracy seule ne suffit donc pas.

- **Matrice de confusion** : pour voir le détail des vrais et faux positifs et négatifs.
- **Recall (sensibilité) sur la classe maligne** : ma métrique principale. Elle mesure la part des cancers réellement détectés.
- **Precision** : la part des alertes « maligne » qui sont justes.
- **F1-score** : le compromis entre precision et recall.
- **Accuracy** : à comparer à la baseline de 63 %, obtenue en prédisant toujours « bénigne ».

Résultats

[À compléter : tableau des métriques du perceptron (30 variables et ACP) et de la régression logistique de comparaison, matrice de confusion.]

Conclusion sur l'efficacité du perceptron

Les deux classes de ce dataset sont presque séparables de façon linéaire, ce qui explique qu'un modèle aussi simple que le perceptron obtienne déjà de bons résultats une fois les données standardisées. Il reste cependant limité pour un usage médical : il ne donne pas de probabilité, ne permet pas de régler le seuil pour favoriser le recall, et son résultat peut varier selon l'ordre des données et le nombre d'époques.

[À ajuster selon mes résultats réels.]

Pistes d'amélioration (non implémentées)

- **Régression logistique** : même idée qu'un perceptron, mais avec une sigmoïde qui donne une probabilité de malignité.
- **Ajuster le seuil de décision** : baisser le seuil pour détecter plus de cancers, quitte à avoir plus de fausses alertes.
- **Perceptron multicouche (MLP)** : ajouter des couches cachées pour capter des relations non linéaires.
- **Autres modèles** : SVM, Random Forest ou gradient boosting, souvent très performants sur ce type de données.
- **Validation croisée** : évaluer le modèle sur plusieurs découpages pour un résultat plus fiable qu'un seul split train/test.
- **Sélection de variables avec Boruta** : garder des variables médicales interprétables plutôt que des composantes ACP.

Bibliographie

- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408.
- Minsky, M. et Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Wolberg, W., Mangasarian, O., Street, N. et Street, W. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository : archive.ics.uci.edu/dataset/17
- scikit-learn-contrib, boruta_py : github.com/scikit-learn-contrib/boruta_py
- Google AI Experiments, Quick, Draw! : experiments.withgoogle.com
- Vidéos de la base de connaissances du sujet : « Perceptron Algorithm with Code Example – ML for beginners! » et « Gradient Descent – Simply Explained! »